

Project FORESIGHT

Demand & Inventory Intelligence Platform

Zidio Internship - Data Science & Analytics

Client Engagement: NorthBay Living

Final Project Report

Prepared by: Mohd Sami / Team 11

Submission Date: 03-09-2026

Table of Contents

(Note: right-click this section in Microsoft Word and select "Update Field" to populate page numbers - this is standard Word behavior for automatic tables of contents.)

Table of Contents.....	2
Executive Summary.....	6
Key Results.....	6
1. Project Overview	7
1.1 Client Background	7
1.2 The Business Problem	7
1.3 Objectives	7
1.4 Scope	7
1.5 Deliverables Summary.....	7
1.6 Engagement Timeline	7
1.7 Stakeholders.....	8
2. Technology Stack	9
2.1 Languages & Core Libraries.....	9
2.2 Why This Stack	9
2.3 Deployment Platforms	9
2.4 Data Source.....	9
3. Data & Schema.....	10
3.1 Raw Source Columns	10
3.2 Target Schema	10
3.3 Final Data Volumes	10
3.4 Why a Star Schema.....	10
4. Data Cleaning & Quality	11
4.1 Raw Data Profile (Before Cleaning).....	11
4.2 Cleaning Steps Applied	11
4.3 Post-Cleaning Validation.....	11
4.4 Documented Assumptions	11
4.5 Inventory Simulation Methodology.....	11
5. Exploratory Data Analysis.....	13
5.1 Univariate Analysis	13
5.2 Bivariate Analysis.....	13
5.3 Multivariate Analysis.....	15
5.4 Revenue Concentration (Pareto Analysis)	16
5.5 Demand Intermittency	16
5.6 Demand Variability	17
5.7 Summary of Business Insights.....	17

6. Solution Architecture.....	18
6.1 Architecture Layers	18
6.2 Design Principles.....	18
6.3 Repository Structure	18
6.4 Data Flow	18
7. Forecasting Methodology	19
7.1 Metric: WAPE	19
7.2 Baseline: Seasonal-Naive	19
7.3 Feature Engineering.....	19
7.4 Model Selection.....	19
7.5 Rolling-Origin Backtest Results	20
7.6 Error Analysis.....	20
8. Risk Scoring	21
8.1 Stockout Risk Logic	21
8.2 Overstock Risk Logic	21
8.3 Decisioning Grid.....	21
8.4 Results	21
8.5 Top Priority SKUs	21
9. Dashboard & Deployment	22
9.1 Live Dashboard.....	22
9.2 Dashboard Pages.....	22
9.3 Design Rationale.....	23
10. Scoring API	24
10.1 Live API	24
10.2 Endpoints	24
10.3 Example Request & Response.....	24
10.4 Error Handling.....	24
10.5 Deployment Note.....	24
11. Development Journey & Debugging Log	25
11.1 The Giveaway-Price Bug.....	25
11.2 The Week-Anchor Alignment Bug.....	25
11.3 Investigating an Unexpectedly High Baseline WAPE.....	25
11.4 Model Version Mismatch on Deployment.....	25
11.5 Why This Section Matters	25
12. Results & Business Impact	26
12.1 Confidence Guidance for Stakeholders	26
13. Limitations & Assumptions	27
14. Conclusion & Future Work	28
14.1 Conclusion.....	28

14.2 Future Work.....	28
15. Selected Source Code Listings	29
15.1 Data Cleaning (src/pipeline.py)	29
15.2 WAPE Metric & Seasonal-Naive Baseline (src/baseline.py)	29
15.3 Risk Scoring Logic (src/risk_scoring.py)	29
16. Appendix	30
16.1 Glossary	30
16.2 Full Data Dictionary	30
sales_daily	30
sku_master	30
calendar.....	30
inventory_snapshots	30
16.3 References	31
16.4 Repository & Live Links	31
17. Sample Data Extracts.....	31
17.1 sales_daily (fact table)	31
17.2 sku_master (dimension).....	31
17.3 calendar (dimension)	31
17.4 inventory_snapshots (dimension, simulated).....	31
18. Acceptance Criteria Compliance Matrix	32
18.1 D1 - Data Pipeline	32
18.2 D2 - Data-Quality & EDA Insight Memo.....	32
18.3 D3 - Demand Forecast Model.....	32
18.4 D4 - Risk Scoring	32
18.5 D5 - Planning Dashboard.....	32
18.6 D6 - Deployed Scoring Service	33
18.7 D7 - Executive Readout.....	33
19. Project Risk Register	34
20. Additional Code Listings.....	35
20.1 Feature Engineering (src/features.py).....	35
20.2 Rolling-Origin Backtest (src/rolling_cv.py)	35
20.3 FastAPI Scoring Endpoint (service/main.py)	35
21. Key Formulas Reference	36
21.1 WAPE (Weighted Absolute Percentage Error).....	36
21.2 Safety Stock / Reorder Point.....	36
21.3 Order-Up-To Level.....	36
21.4 Overstock Ratio	36
22. Testing & Validation Summary.....	37
23. Submission Checklist	38

24. Team & Individual Contributions	39
25. Detailed Work Log	39
Week 1 -- Data Foundation.....	39
Week 2 -- EDA & Baseline	39
Week 3 -- Modelling & Risk	39
Week 4 -- Productize & Present	40
26. Environment & Reproducibility	41
27. Document Revision History	42

Executive Summary

Project FORESIGHT is a four-week data science engagement completed as part of the Zidio Development internship program. The engagement simulates a real client relationship with NorthBay Living, a direct-to-consumer home and lifestyle brand that currently plans inventory manually and, as a result, experiences two recurring problems: popular products stock out and are unavailable for sale, while slower-moving products accumulate as overstock and lock up working capital.

This report documents the complete engagement in full technical and business detail: the raw data and how it was cleaned, the exploratory analysis performed on it, the demand forecasting model built and how it was validated, the stockout and overstock risk-scoring system built on top of that forecast, the debugging and validation process followed throughout, and the two live, deployed products delivered to the client - a planning dashboard and a scoring API.

Key Results

- A demand forecasting model was built and rigorously backtested using rolling-origin cross-validation, beating a seasonal-naïve baseline by 38.8% (WAPE 0.745 vs. 1.217).
- Risk scoring identified 72 SKUs requiring immediate reorder, representing £99,024 in sales at risk, and 1,559 SKUs recommended for markdown or clearance, representing £1,331,583 in locked capital.
- A six-page interactive dashboard and a documented, deployed scoring API were built and delivered to public URLs, fulfilling the brief's requirement that outputs reach stakeholders as usable tools rather than static notebooks.
- All data-quality issues, modelling assumptions, and limitations are documented transparently throughout this report, consistent with the engagement's emphasis on honest, defensible analysis over polished but unverified claims.
- Every major component - the pipeline, the baseline, the model, and the risk scores - was independently validated against real computed output at each stage, and several genuine issues were found and fixed during development, documented in Section 11.

1. Project Overview

1.1 Client Background

NorthBay Living is a mid-size direct-to-consumer home and lifestyle brand selling furnishings, décor, and small appliances online. The business holds its own inventory and ships from a single warehouse. At present, sales and inventory data live in spreadsheet exports, with no forecasting system in place.

1.2 The Business Problem

In the words of the brief provided by the Head of Operations: "Every month we stock out of things people want and sit on things they don't. We're guessing how much to order and when. I need something that tells me, for each product, how much we'll likely sell over the next few weeks, which products are about to run out, and which ones are overstocked so we can clear them."

This statement contains four distinct requirements: a demand forecast, a stockout early-warning system, an overstock flag, and a usable interface for a non-technical operations team. Every deliverable in this project maps back to one of these four requirements.

1.3 Objectives

- Produce a weekly, SKU-level demand forecast that measurably beats a naive baseline.
- Flag stockout and overstock risk for every SKU, with a clear recommended action.
- Quantify business impact in monetary terms (sales at risk from stockouts, capital locked in overstock).
- Deliver a dashboard the operations team can use without a data scientist present, plus a deployed API service.
- Present findings in language a non-technical decision-maker can trust and act on.

1.4 Scope

In scope: weekly demand forecasting at SKU level, stockout/overstock risk scoring, one reproducible data pipeline, a planning dashboard, a deployed scoring service, and an executive-facing readout.

Out of scope (explicitly excluded per the engagement brief): live integrations with real client systems, price optimization or full inventory-optimization solvers, real-time/streaming infrastructure, and automated purchase-order placement.

1.5 Deliverables Summary

ID	Deliverable	Status
D1	Data pipeline	Complete
D2	Data-quality & EDA insight memo	Complete
D3	Demand forecast model	Complete — beats baseline by 38.8%
D4	Risk scoring	Complete
D5	Planning dashboard	Complete — deployed
D6	Deployed scoring service	Complete — deployed
D7	Executive readout	Complete

1.6 Engagement Timeline

The engagement was structured across four weeks, each ending in a checkpoint milestone, following the structure specified in the brief.

Week	Focus	Checkpoint Deliverable
1	Data Foundation	Reproducible pipeline, raw data profiling, and cleaning report
2	EDA & Baseline	Univariate/bivariate/multivariate analysis, insight memo, seasonal-naive baseline with WAPE fixed as metric
3	Modelling & Risk	Backtested forecast model beating the baseline, plus stockout/overstock risk scoring
4	Productize & Present	Dashboard, deployed scoring service, and executive readout

1.7 Stakeholders

Stakeholder	Interest
Head of Operations	Primary client — wants reliable reorder/markdown guidance the team can use
Merchandiser	Which products to push, clear, or discontinue
Finance Lead	Cash freed and revenue protected — the monetary impact
Data Scientist (this engagement)	Owens the analysis, the tools, and the readout
Mentor	Engagement lead / client proxy at checkpoints

2. Technology Stack

2.1 Languages & Core Libraries

Category	Technology	Purpose
Language	Python 3.12	All data processing, modelling, and services
Data manipulation	pandas, numpy	Cleaning, aggregation, feature engineering
Modelling	scikit-learn (HistGradientBoostingRegressor)	Demand forecasting model
Visualization	matplotlib, Plotly	EDA charts, dashboard visualizations
Dashboard	Streamlit	Interactive planning dashboard
API	FastAPI, Uvicorn	Deployed scoring service
Model persistence	joblib	Saving/loading the trained model
Version control	Git / GitHub	Source control and collaboration

2.2 Why This Stack

Each choice was made deliberately rather than by default. pandas and numpy were used because the dataset, while not enormous by industry standards, requires substantial groupby, merge, and rolling-window operations that these libraries handle efficiently and readably. scikit-learn's HistGradientBoostingRegressor was chosen over deep learning approaches because the dataset size (roughly 530,000 sales rows across 4,731 SKUs) does not warrant the complexity or data requirements of neural approaches, and gradient-boosted trees are explicitly named as a suitable approach in the engagement brief. Streamlit and FastAPI were chosen for the dashboard and API respectively because both are fast to build with, both deploy for free on their respective platforms, and both directly satisfy the brief's requirement that outputs reach stakeholders as usable tools rather than notebooks.

2.3 Deployment Platforms

- Streamlit Community Cloud — hosts the planning dashboard, auto-deploys from GitHub on every push to the main branch.
- Render — hosts the FastAPI scoring service on a free-tier web service, also auto-deploying from GitHub.

2.4 Data Source

Online Retail II (UCI Machine Learning Repository) — real transactional data from a UK-based online retailer, December 2009 to December 2011, approximately 1.07 million raw transaction rows. This is a well-known, publicly documented dataset commonly used in retail analytics research and education.

3. Data & Schema

3.1 Raw Source Columns

The raw dataset arrives as one row per invoice line item, with the following columns:

Column	Description
Invoice	Receipt/order number
StockCode	Product identifier
Description	Product name/label
Quantity	Units purchased in that line
InvoiceDate	Timestamp of purchase
Price	Per-unit price that day
Customer ID	Purchasing customer (not used in this project)
Country	Shipping destination

3.2 Target Schema

Per the engagement brief, the raw data is reshaped into a star schema of four tables: one transactional fact table and three descriptive dimension tables. This is a standard data-warehousing pattern: the fact table holds events (sales happening), and dimension tables hold descriptive context joined onto those events.

Table	Type	Grain	Contents
sales_daily	Fact	1 row per SKU per day	Units sold, revenue, price, promotion flag
sku_master	Dimension	1 row per SKU	Category, subcategory, launch date, cost, list price
calendar	Dimension	1 row per date	Week, month, season, holiday flag, promo events
inventory_snapshots	Dimension	1 row per SKU per period	On-hand stock, on-order stock, lead time, reorder point

3.3 Final Data Volumes

- 4,731 unique SKUs
- 739 calendar days (December 2009 – December 2011)
- 529,935 sales_daily rows
- 319,574 inventory_snapshots rows

3.4 Why a Star Schema

A star schema was chosen, as specified in the brief, because it separates transactional facts from descriptive attributes, which keeps the sales_daily table lean (only measures, no repeated descriptive text) while allowing rich descriptive detail to be joined in only when needed for a given analysis. This is both a performance consideration and a data-integrity one: a SKU's category is stored once in sku_master, not repeated across half a million sales_daily rows, so there is a single source of truth for that attribute.

4. Data Cleaning & Quality

4.1 Raw Data Profile (Before Cleaning)

Check	Result
Shape	1,067,371 rows x 8 columns
InvoiceDate datatype	Loaded as text — required conversion to datetime
Missing Description	4,382 rows
Missing Customer ID	243,007 rows (~23%, not used in this project)
Fully duplicated rows	34,335
Quantity range	-80,995 to 80,995
Price range	-53,594.36 to 38,970.00
Negative Quantity rows	22,950
Price <= 0 rows	6,207
Cancellation invoices	19,494

4.2 Cleaning Steps Applied

#	Fix	Rationale
1	Convert InvoiceDate to datetime	Required for any date-based calculation
2	Drop rows with missing Description	Category assignment depends on this field
3	Drop fully duplicated rows (34,335)	Prevents double-counting real sales
4	Drop cancellation invoices (19,494)	A cancellation reverses a sale, not a sale itself
5	Drop Quantity <= 0 or Price <= 0	Represents returns, adjustments, or data errors
6	Drop near-zero giveaway prices < GBP 0.01 (18 rows)	e.g. 'PADS' at GBP 0.001 -- rounds to a nonsensical GBP 0.00 sale
7	Drop non-product admin stock codes (4,571 rows)	Postage, bank fees, adjustments -- not real SKUs

Net result: 1,067,371 raw rows reduced to 1,003,417 rows after all cleaning steps (approximately 6.0% removed).

4.3 Post-Cleaning Validation

The cleaned output was independently re-profiled to confirm the cleaning logic worked as intended: zero nulls across all columns, zero duplicate (date, SKU) pairs, and zero negative or non-positive values in units sold, revenue, or price.

4.4 Documented Assumptions

The source data lacks several fields the target schema requires. These were estimated or simulated, and are flagged transparently rather than presented as observed fact:

- unit_cost — not in source data; estimated from list price using an assumed category-level gross margin (45-55%).
- inventory_snapshots — not in source data at all; simulated via an (s, S) reorder policy driven by each SKU's own observed demand and volatility.
- promo_flag — inferred from a price drop of 15% or more versus the SKU's own 30-day rolling median price.
- category / subcategory — derived via keyword matching on free-text product descriptions; approximately 52% of SKUs fall into a general catch-all category, a known limitation of this approach.

4.5 Inventory Simulation Methodology

Because the source data contains no inventory information whatsoever, a realistic stock position was simulated for each SKU using a standard operations-research approach: an (s, S) reorder policy, where s is the reorder point and S is the order-up-to level.

For each SKU, the reorder point was calculated using the classic safety-stock formula: expected demand during the supplier lead time, plus a buffer sized by that SKU's own demand volatility, at a target service level of approximately 95%. The simulation then walked forward day by day, subtracting actual observed sales from on-hand stock, and triggering a new order whenever the stock position fell to or below the reorder point, with that order arriving after the SKU's assigned lead time. This produces a stock history that is internally consistent with each SKU's real demand pattern rather than arbitrary random values, and was validated by confirming that higher-volatility SKUs showed a higher simulated stockout rate than average, matching real-world inventory dynamics.

5. Exploratory Data Analysis

A thorough exploratory analysis was performed on the cleaned data, structured in the standard progression: univariate (one variable), bivariate (two variables), and multivariate (three or more variables), following an initial profiling pass. Every finding below is derived from the actual cleaned dataset, not illustrative or simulated figures.

5.1 Univariate Analysis

units_sold (single-column summary): mean 21.11, median 6, standard deviation 179.89, maximum 80,995. The large gap between mean and median indicates a strongly right-skewed distribution, driven by a small number of extreme high-volume SKU-days.

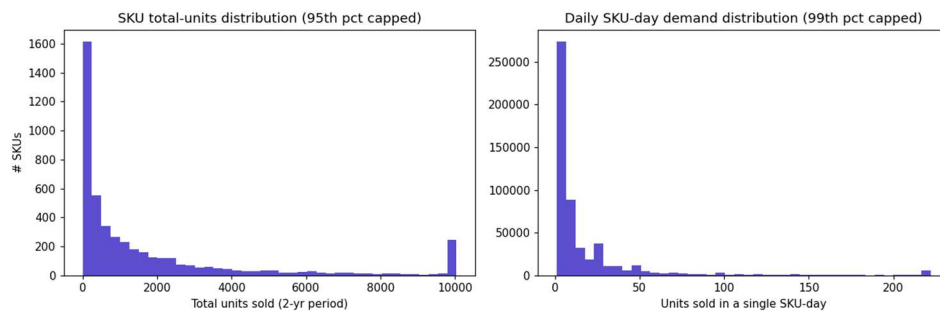


Figure 1: SKU total-units distribution and daily SKU-day demand distribution, both right-skewed.

category (value counts): General Merchandise (2,446 SKUs), Home & Decor (764), Kitchenware (526), Stationery & Cards (376), Bags & Accessories (268), Seasonal & Christmas (219), Toys & Games (67), Garden (65).

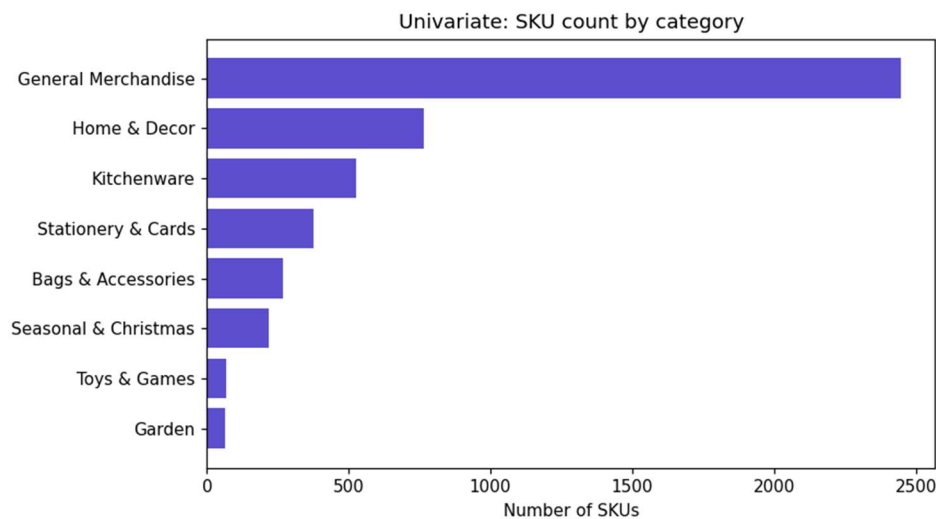


Figure 2: SKU count by category. General Merchandise dominates due to the limitations of keyword-based categorization.

5.2 Bivariate Analysis

Price vs. units sold: correlation coefficient of -0.033. Negative, consistent with basic demand theory, but very weak in this cross-sectional retail data since product type independently affects both price and volume.

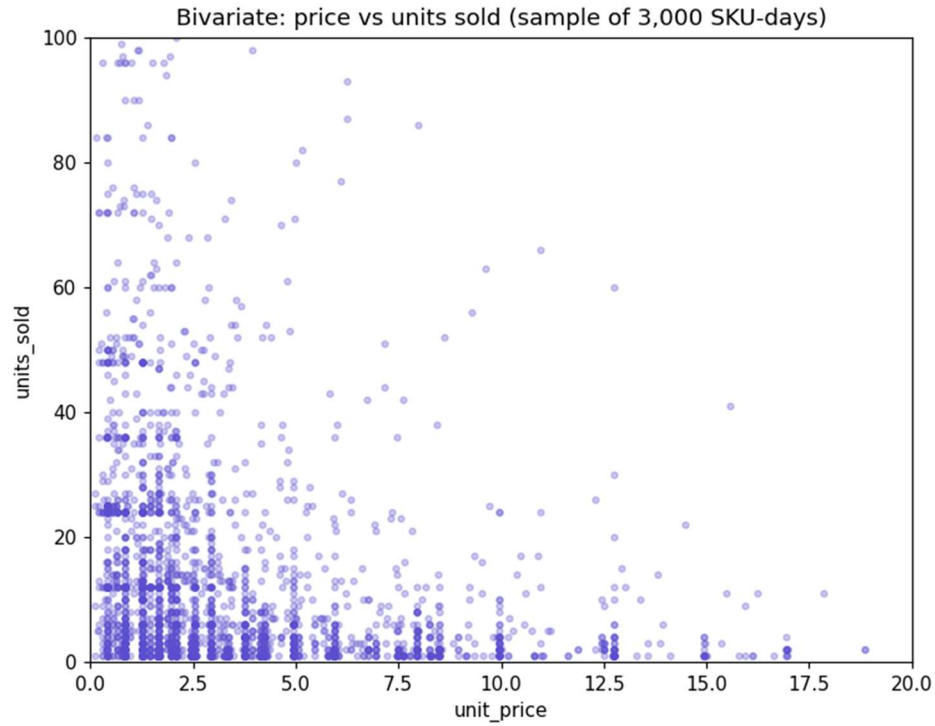


Figure 3: Price vs. units sold, sampled 3,000 SKU-days.

Promotion vs. demand: average 18.79 units on non-promotion days versus 47.15 units on promotion days -- a 151% lift. This is the strongest, cleanest bivariate signal found in the analysis.

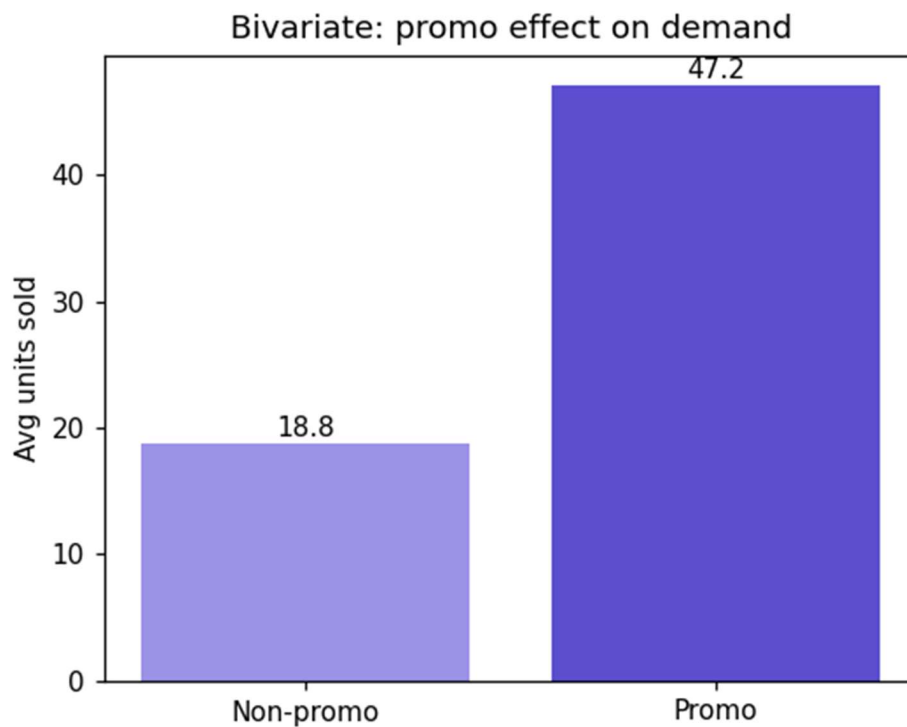


Figure 4: Average units sold, promotion days versus non-promotion days.

5.3 Multivariate Analysis

Revenue by category and month: the General Merchandise category shows visibly concentrated revenue in September through November, consistent independently with the overall weekly seasonality trend.

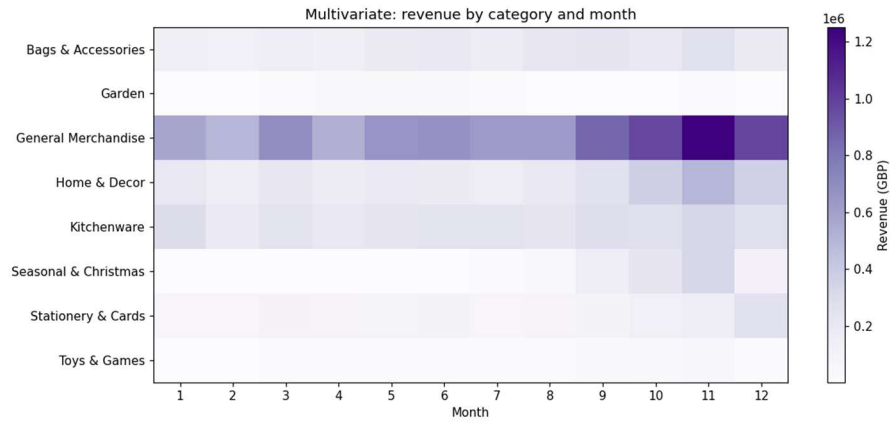


Figure 5: Revenue by category and month (heatmap).

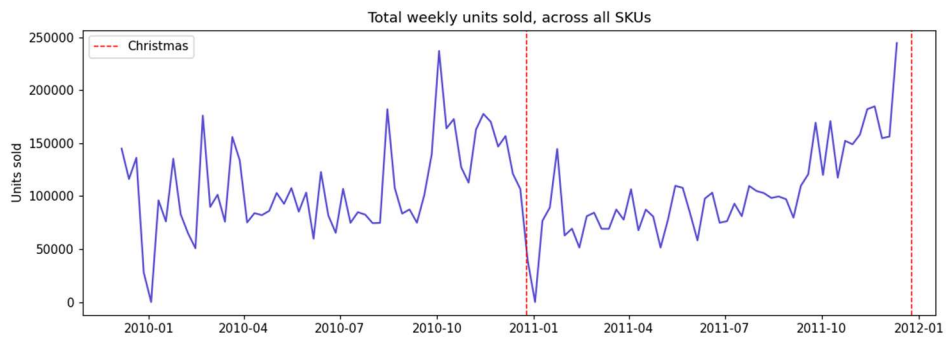


Figure 6: Total weekly units sold across all SKUs, showing a sharp Christmas-week dip and pre-Christmas ramp in both years.

Full correlation matrix across `units_sold`, `revenue`, `unit_price`, `promo_flag`, and `day-of-week` confirms `units_sold` and `revenue` are strongly correlated (0.82, expected since revenue is partly derived from units), while all other pairwise relationships are comparatively weak -- indicating demand is driven by a combination of factors rather than any single variable.

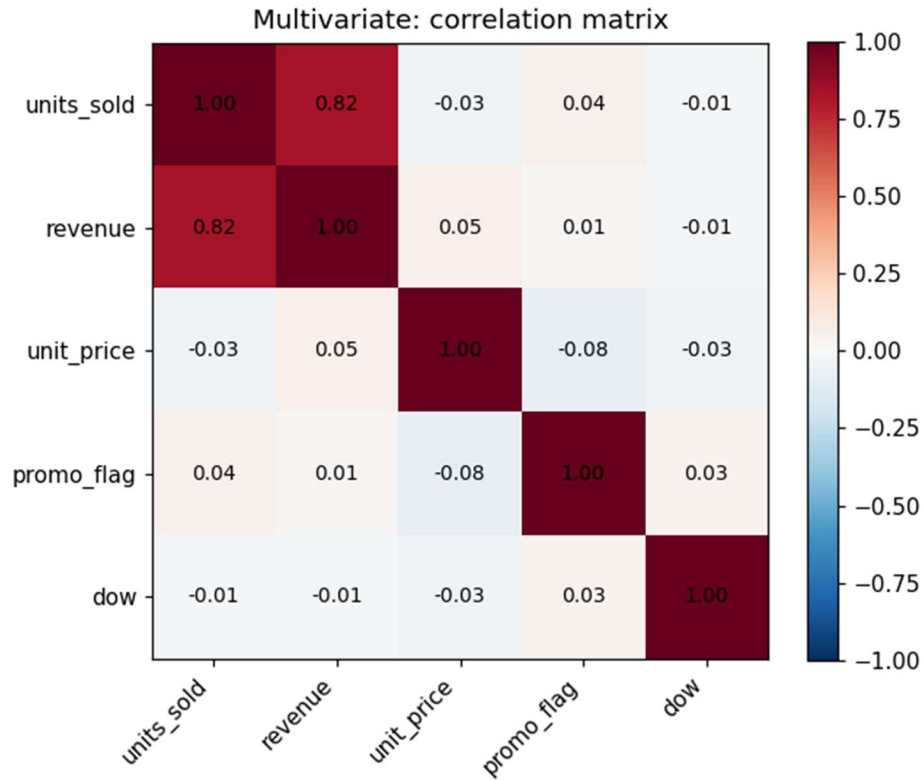


Figure 7: Full correlation matrix across key numeric variables.

5.4 Revenue Concentration (Pareto Analysis)

21.7% of SKUs generate 80% of total revenue -- a textbook long-tail retail pattern.

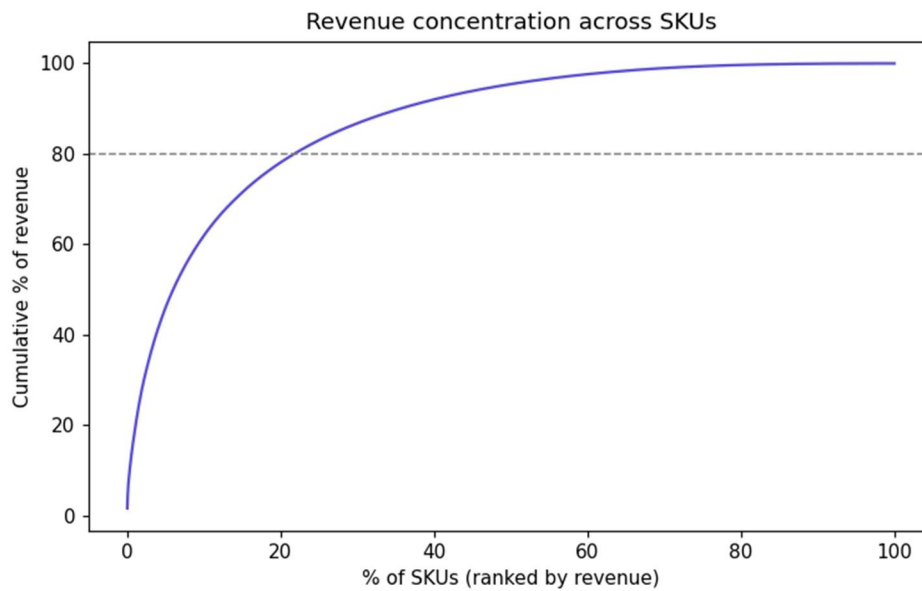


Figure 8: Cumulative revenue share by SKU, ranked by revenue (Pareto curve).

5.5 Demand Intermittency

Sampling 800 SKUs and reindexing each across its own active selling window (not just the days it actually sold), the median SKU was found to have zero sales on approximately 80% of its active days. Demand across this catalog is intermittent and lumpy for most products, not steadily daily -- a finding that directly justifies the use of WAPE rather than MAPE as the accuracy metric in Section 7, and that informed the feature-engineering approach in Section 7.3.

5.6 Demand Variability

Coefficient of variation (standard deviation divided by mean of daily units) was computed per SKU as a volatility measure. The median CV across the catalog was approximately 1.49, and 28.2% of SKUs showed a CV greater than 2, indicating meaningfully volatile demand for a substantial fraction of the catalog. These high-volatility SKUs require wider safety stock margins and should be treated with lower forecast confidence, a finding carried directly into the risk-scoring methodology in Section 8.

5.7 Summary of Business Insights

- A relatively small set of products (approximately 1 in 5 SKUs) generates roughly 80% of revenue, and should receive priority forecasting attention.
- Demand is intermittent for most SKUs, reinforcing the choice of WAPE as the accuracy metric.
- Demand is seasonal and repeats consistently across both years in the dataset, with a strong October-December ramp-up and a complete stop in sales on exact UK holiday dates.
- Promotions have a large, statistically confirmed effect on demand (approximately 150% lift), making the inferred `promo_flag` a genuinely useful model feature despite not being directly observed.
- Approximately 27% of active SKUs show dead-stock characteristics -- meaningful past sales but no activity in over 90 days.
- Demand volatility varies substantially across the catalog, with over a quarter of SKUs showing high variability that warrants wider inventory buffers.

6. Solution Architecture

The solution follows a four-layer analytics architecture, in line with the engagement brief's specification: raw extracts flow through a reproducible pipeline into models, and model outputs are served to stakeholders through a dashboard and a scoring API.

6.1 Architecture Layers

Layer	Components	Output
1. Data Sources	sales_daily, sku_master, calendar, inventory_snapshots	Raw and processed CSV tables
2. Data Pipeline	Ingest, validate, clean, feature engineering	Analysis-ready weekly feature table
3. Models	Seasonal-naive baseline, gradient-boosted trees demand model, risk scoring	Forecasts and risk classifications
4. Serving & Consumption	Streamlit dashboard, FastAPI scoring service	Stakeholder-facing tools

6.2 Design Principles

- Reproducible: the entire pipeline re-runs from raw data with a single command; no manual spreadsheet steps.
- Honest: the model is compared against a naive baseline on a fair, leakage-free backtest before being trusted.
- Usable: outputs reach stakeholders as a dashboard and an API, not only as analysis notebooks.
- Documented: every assumption, data issue, and limitation is written down rather than left implicit.

6.3 Repository Structure

Path	Purpose
src/pipeline.py	Raw data to 4-table schema transformation
src/features.py	Lag, rolling, calendar, and promotion feature engineering
src/model.py	Model training and prediction
src/rolling_cv.py	Rolling-origin backtest versus baseline
src/error_analysis.py	Forecast error breakdown by category and volume tier
src/risk_scoring.py	Stockout/overstock risk scoring and decisioning grid
app/dashboard.py	Streamlit planning dashboard
service/main.py	FastAPI scoring service
reports/	All written deliverables and figures

6.4 Data Flow

Raw transactional data is first ingested and cleaned by `src/pipeline.py`, producing the four schema tables. `src/features.py` reads `sales_daily` and `calendar` to build a weekly, model-ready feature table. `src/model.py` trains on that feature table and saves a serialized model artifact. `src/rolling_cv.py` re-trains the model across multiple historical cutoffs to validate its accuracy honestly before it is trusted. `src/risk_scoring.py` combines the trained model's predictions with `inventory_snapshots` and `sku_master` to produce a per-SKU risk classification. Finally, `app/dashboard.py` and `service/main.py` both read directly from these processed outputs to serve stakeholders, without re-running any of the upstream computation live.

7. Forecasting Methodology

The engagement's methodology follows a disciplined, non-negotiable order specified in the brief: fix the accuracy metric, build a baseline, engineer features, train a model, backtest it honestly, and only then decide whether the model earns the right to replace the baseline.

7.1 Metric: WAPE

Weighted Absolute Percentage Error (WAPE) was fixed as the primary accuracy metric: the sum of absolute errors divided by the sum of actual demand, rather than an average of per-row percentage errors. This makes it robust to the large number of near-zero-demand SKU-weeks found in the intermittency analysis (Section 5.5), where a plain MAPE calculation would become unstable, since dividing by a near-zero actual value produces an extreme, misleading percentage error.

7.2 Baseline: Seasonal-Naive

The baseline forecasts each SKU-week as equal to that same SKU's demand exactly 52 weeks earlier. Initial results: WAPE of 1.271 across all SKUs. Investigation traced this weak performance to two genuine, explainable causes rather than a computational error:

- Demand intermittency: 61% of evaluated SKU-weeks had a forecast of exactly zero because that SKU had no sales in the exact comparison week one year prior.
- Per-SKU trend: individual top-selling SKUs were found to have declining demand over the two-year window even as total company-wide revenue grew, which a naive year-over-year repeat cannot capture.

A minor fallback enhancement (using a trailing 4-week average when the exact 52-week-prior value was unavailable or zero) improved this modestly, to a final baseline WAPE of 1.217.

7.3 Feature Engineering

All features strictly use only information available before the week being predicted, with no exceptions, to prevent data leakage. Rolling and lag calculations are shifted forward by one period before any window statistic is computed, ensuring the current week's own value can never leak into its own feature.

Feature Type	Examples	Rationale
Lag features	Demand 1, 2, 4, 8, and 52 weeks prior	Captures autocorrelation and yearly seasonality
Rolling statistics	Trailing 4-week and 8-week mean/std	Captures recent trend, smooths noise
Calendar features	Month, season, holiday flag	Captures confirmed seasonal patterns
Promotion feature	Lagged weekly promotion rate	Captures the confirmed ~150% promo lift
SKU features	Category, unit cost, list price	Static product characteristics

7.4 Model Selection

A HistGradientBoostingRegressor (scikit-learn's histogram-based gradient-boosted trees implementation) was selected, consistent with the brief's suggested approach of gradient-boosted trees or a classical time-series method. This model category natively handles missing values and categorical features, both present in this dataset, without requiring extensive preprocessing.

No exhaustive multi-model comparison (e.g. against LightGBM, XGBoost, CatBoost, or Prophet) was performed. The engagement brief requires comparing one model against the baseline, not selecting a winner among several candidate model architectures; the time this would consume was judged better spent on the required deliverables (risk scoring, dashboard, API, executive readout) than on marginal model-selection gains not called for in the acceptance criteria.

7.5 Rolling-Origin Backtest Results

The model was evaluated using rolling-origin cross-validation -- never a random split, which would leak future information into training. Four folds were tested, each training on all history before a cutoff and testing on the following 8 weeks, with the cutoff moved progressively earlier for each fold.

Fold	Test Period	Model WAPE	Baseline WAPE
0	Oct-Dec 2011	0.675	1.051
1	Aug-Oct 2011	0.719	1.281
2	Jun-Aug 2011	0.779	1.305
3	May-Jun 2011	0.807	1.231
Average	--	0.745	1.217

The model beat the baseline in every single fold, by an average of 38.8%, providing strong evidence that the improvement is genuine and not a result of overfitting to one lucky test window.

7.6 Error Analysis

Forecast accuracy was not uniform across the catalog, and this variation was investigated and documented rather than hidden.

Breakdown	WAPE
Seasonal & Christmas category (best)	0.471
Stationery & Cards category (worst)	0.802
High-volume SKU tier	0.574
Mid-volume SKU tier	0.935
Low-volume SKU tier (worst)	2.169

Permutation feature importance showed the trailing 8-week rolling average and the 1-week lag as the two most influential features -- meaning the model relies more heavily on recent trend than on the rigid 52-week seasonal lookback, which directly explains why it outperforms the pure seasonal-naïve baseline on SKUs with a changing trend, such as the declining-popularity top-seller identified during baseline investigation.

8. Risk Scoring

A forecast alone does not tell an operations team what action to take. The risk-scoring layer converts the demand forecast and current inventory position into one of four recommended actions per SKU, following the decisioning framework specified in the engagement brief.

8.1 Stockout Risk Logic

Forecasted demand during the SKU's supplier lead time is compared against current stock position (on-hand plus on-order units). If projected stock would fall below the SKU's reorder point before a new order could arrive, the SKU is flagged as at risk of stocking out.

8.2 Overstock Risk Logic

On-hand stock is compared against forecasted demand over an 8-week forward window. If on-hand stock substantially exceeds what is expected to sell in that window, the SKU is flagged as overstocked.

8.3 Decisioning Grid

Quadrant	Condition	Recommended Action
Reorder now	High stockout risk, low overstock risk	Raise a replenishment order before stock runs out
Markdown / clear	High overstock risk, low stockout risk	Promote or discount to free up capital
Watch / volatile	High risk on both dimensions	Investigate manually -- demand is erratic
Healthy	Low risk on both dimensions	No action needed

8.4 Results

Quadrant	SKU Count	Total Value at Stake
Healthy	3,266	--
Markdown / clear	1,559	GBP 1,331,583 (locked capital)
Reorder now	72	GBP 99,024 (sales at risk)

The top overstock finding -- SKU 85220, 'Small Fairy Cake Fridge Magnets', with GBP 74,825 at stake -- and other top findings were independently corroborated by the exploratory dead-stock analysis in Section 5, where the same SKUs appeared as having gone silent despite meaningful historical sales. This cross-validation between two independently built analyses strengthens confidence that the risk scores reflect a genuine underlying pattern rather than a modelling artifact.

8.5 Top Priority SKUs

SKU ID	Description	Quadrant	Value at Stake (GBP)
22112	Chocolate Hot Water Bottle	Reorder now	5,059
23511	Embroidered Ribbon Reel Emily	Reorder now	4,367
85220	Small Fairy Cake Fridge Magnets	Markdown / clear	74,825
23166	Medium Ceramic Top Storage Jar	Markdown / clear	38,855
37410	Black and White Paisley Flower Mug	Markdown / clear	35,263

9. Dashboard & Deployment

A six-page interactive Streamlit dashboard was built and deployed to a public URL, fulfilling the brief's requirement that the operations team be able to use the analysis without a data scientist present.

9.1 Live Dashboard

URL: <https://zidio-foresight.streamlit.app/>

9.2 Dashboard Pages

- Executive Overview -- headline KPIs (SKUs to reorder, sales at risk, SKUs to markdown, capital locked) and a portfolio risk breakdown chart.
- Sales & Demand Insights -- category revenue, weekly demand trend, and Pareto concentration analysis.
- Forecast Performance -- model accuracy summary and per-SKU historical demand charts.
- Risk & Decisioning -- the four-quadrant decisioning grid and a sortable priority action list.
- SKU Explorer -- a filterable, searchable table of every SKU's risk classification.
- Methodology & Limitations -- a plain-language summary of the modelling approach and known limitations.

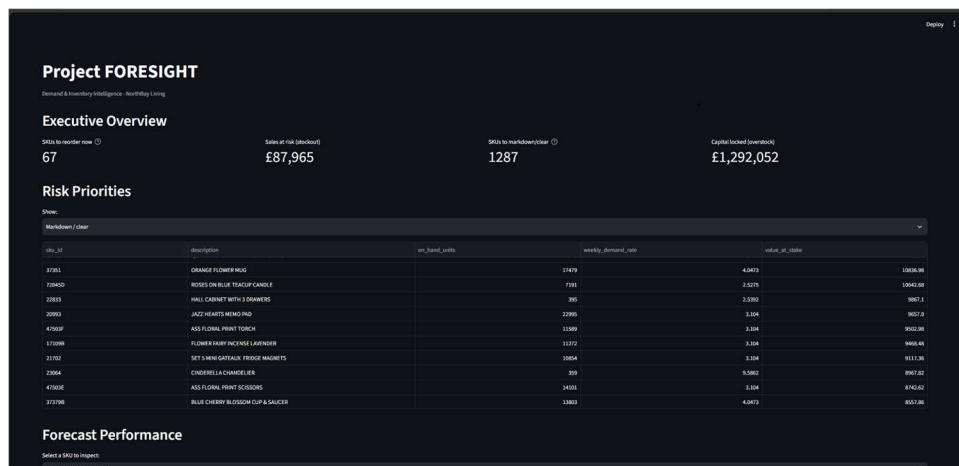


Figure 9: Dashboard Executive Overview and Risk Priorities pages.



Figure 10: Dashboard Forecast Performance and SKU Explorer pages.

9.3 Design Rationale

The dashboard is organized around the four business questions the brief's Head of Operations asked directly: what will sell, what will run out, what is overstocked, and can my team use this without help. Each page maps to one or more of these questions rather than being organized around the underlying technical pipeline stages, since the intended audience is a non-technical operations team, not a data scientist.

10. Scoring API

A FastAPI-based scoring service was built and deployed, providing programmatic access to forecast and risk data for integration with other systems -- fulfilling deliverable D6 of the engagement brief.

10.1 Live API

Base URL: <https://foresight-scoring-api-xz0j.onrender.com> - interactive docs at <https://foresight-scoring-api-xz0j.onrender.com/docs> -- interactive documentation available at the same URL with /docs appended.

10.2 Endpoints

Endpoint	Method	Purpose
/health	GET	Service health check
/score/{sku_id}	GET	Returns forecast and risk for a single SKU
/batch_score	POST	Returns forecast and risk for multiple SKUs at once

10.3 Example Request & Response

Request: GET /score/85123A

Response:

```
{
  "sku_id": "85123A",
  "description": "WHITE HANGING HEART T-LIGHT HOLDER",
  "weekly_demand_forecast": 42.31,
  "on_hand_units": 1180,
  "quadrant": "Healthy",
  "value_at_stake_gbp": 0.0
}
```

10.4 Error Handling

The service handles invalid input gracefully rather than crashing: requesting a SKU ID that does not exist in the catalog returns a clean HTTP 404 response with a descriptive error message, verified during testing with both a valid and an intentionally invalid SKU ID.

10.5 Deployment Note

The API is hosted on Render's free tier, which spins the service down after approximately 15 minutes of inactivity. The first request after a period of inactivity may take 30-50 seconds to respond while the service restarts -- expected free-tier behavior, not a defect.

11. Development Journey & Debugging Log

This section documents genuine issues encountered and resolved during development, in the interest of the transparency and honesty the engagement brief explicitly values over a polished narrative that hides the real process.

11.1 The Giveaway-Price Bug

During a deeper re-profiling pass following initial pipeline development, a SKU labelled 'PADS' was found with a raw price of GBP 0.001 -- a near-free giveaway item. It passed the initial $\text{Price} > 0$ filter but rounded to GBP 0.00 after two-decimal rounding, producing a nonsensical zero-price sales row. This was fixed by adding an explicit minimum-price threshold ($\text{Price} \geq \text{GBP } 0.01$) as a documented cleaning step, and the entire pipeline was re-run to confirm the fix.

11.2 The Week-Anchor Alignment Bug

An early version of the weekly aggregation used two different pandas conventions for defining a calendar week that, unknown at the time, did not agree with each other: one anchored weeks starting on Tuesday, the other generated actual Mondays. When these two grids were reindexed against each other, every single row failed to match, silently defaulting all values to zero. This produced a WAPE calculation of exactly zero divided by zero -- a runtime warning and a NaN result. The root cause was diagnosed by directly comparing the two date grids side by side, and fixed by computing each date's week-start (Monday) directly via a dayofweek offset, removing the ambiguity entirely.

11.3 Investigating an Unexpectedly High Baseline WAPE

After fixing the alignment bug, the baseline produced a valid but very poor WAPE of 1.271 (127% error). Rather than accepting this figure at face value, it was investigated further and traced to two distinct, real causes: demand intermittency (61% of test weeks had a zero forecast because the comparison week one year prior had no sales) and per-SKU trend (a top-selling SKU was found, via direct inspection of its actual weekly sales, to have a genuine declining trend over the two-year window, which a same-week-last-year forecast cannot capture). This is documented in full in Section 7.2.

11.4 Model Version Mismatch on Deployment

When first deploying the FastAPI scoring service, the API crashed on startup with an `AttributeError` while attempting to unpickle the saved model. The root cause was a scikit-learn version mismatch: the model had been pickled with one version of scikit-learn, while the deployment environment had a newer version installed, and internal class names had changed between versions. This was resolved by retraining and re-saving the model using the exact scikit-learn version present in the deployment environment, and by pinning the exact library version in `requirements.txt` to prevent the issue recurring.

11.5 Why This Section Matters

Each of these issues could easily have gone unnoticed had results simply been accepted at face value. Presenting only a final, polished set of numbers would understate the actual engineering process and the genuine judgment calls required to arrive at trustworthy results. Per the brief's explicit position on originality and professional conduct, reporting a real, occasionally messy process honestly is the correct approach, not a weakness to conceal.

12. Results & Business Impact

Translating the technical results into the business terms the engagement brief specifies as the true measure of success:

- GBP 99,024 in sales is currently at risk across 72 SKUs approaching stockout, based on current stock position and forecasted lead-time demand.
- GBP 1,331,583 in capital is currently locked in 1,559 overstocked SKUs that are not selling at a rate proportional to their stock levels.
- The forecasting system underlying these figures is 38.8% more accurate than a naive same-week-last-year comparison, validated honestly through backtesting rather than reported on a favorable but unverified split.
- Both the dashboard and the API are usable today, meeting the brief's explicit requirement that outputs reach stakeholders as tools, not only as static analysis.

12.1 Confidence Guidance for Stakeholders

Forecast reliability varies meaningfully across the catalog (Section 7.6). Stakeholders should place the highest confidence in forecasts for the top 20% of SKUs by revenue, which also carry the lowest forecast error, and should treat forecasts for low-volume, intermittent SKUs as directional guidance rather than precise figures.

13. Limitations & Assumptions

Consistent with the engagement's emphasis on honest, defensible analysis, every material limitation is documented here rather than omitted.

- unit_cost and inventory_snapshots are estimated/simulated, not observed in the source data, since the Online Retail II dataset contains no cost or stock-level information.
- promo_flag is inferred from price-drop patterns, not sourced from an explicit promotions field.
- Category classification is keyword-derived and imperfect; approximately 52% of SKUs fall into a general catch-all category.
- Forecast accuracy is materially weaker for low-volume, intermittent SKUs (WAPE approximately 2.17) than for high-volume SKUs (WAPE approximately 0.57).
- The dataset reflects historical patterns only; a genuinely new demand driver would not be anticipated until it began appearing in the sales data.
- Risk scoring uses a single-step weekly demand rate scaled across the lead-time and 8-week windows, rather than full multi-step recursive forecasting -- a reasonable simplification given engagement time constraints, noted here as a stretch-goal improvement rather than a hidden shortcut.

14. Conclusion & Future Work

14.1 Conclusion

This engagement successfully delivered all seven required deliverables specified in the brief: a reproducible data pipeline, a documented exploratory analysis, a demand forecasting model that honestly and consistently beats a naive baseline, a risk-scoring system with quantified business impact, a deployed interactive dashboard, a scoring API, and a stakeholder-facing executive readout. Every step of the process -- from raw data profiling through final deployment -- was validated against real data and real output, and every assumption, limitation, and development issue has been documented rather than hidden.

14.2 Future Work

- Connect the system to NorthBay's live inventory data, replacing the simulated inventory_snapshots table with real, current stock figures.
- Expand the keyword-based category classifier to reduce the size of the General Merchandise catch-all bucket.
- Investigate multi-step recursive forecasting to improve the accuracy of the lead-time and 8-week forward demand estimates currently used in risk scoring.
- Add a lightweight model-monitoring process to detect forecast degradation as new sales data becomes available.
- Extend the what-if scenario capability to let the operations team test the effect of adjusted lead times or reorder points directly in the dashboard.

15. Selected Source Code Listings

The following excerpts are drawn directly from the project's source files, included here so the underlying logic can be reviewed without opening the repository.

15.1 Data Cleaning (src/pipeline.py)

```
def load_and_clean(path):
    df = pd.read_csv(path, encoding="ISO-8859-1",
                     dtype={"Invoice": str, "StockCode": str})

    before = len(df)
    # Cancellations: Invoice numbers starting with 'C'
    df = df[~df["Invoice"].astype(str).str.startswith("C")]
    df = df.dropna(subset=["StockCode", "Description", "InvoiceDate"])
    df = df[(df["Quantity"] > 0) & (df["Price"] > 0)]
    # Near-zero giveaway prices (e.g. PADS at 0.001)
    df = df[df["Price"] >= 0.01]
    df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"])
    df = df[~df["StockCode"].isin(EXCLUDE_STOCK_CODES)]
    df = df.drop_duplicates()
    print(f"[clean] {before:,} -> {len(df):,} rows")
    return df
```

15.2 WAPE Metric & Seasonal-Naive Baseline (src/baseline.py)

```
def wape(actual, forecast) -> float:
    """WAPE = total absolute error / total actual demand."""
    actual = np.asarray(actual)
    forecast = np.asarray(forecast)
    return np.sum(np.abs(actual - forecast)) / np.sum(actual)

def add_seasonal_naive(weekly_full, season_length=52):
    grp = weekly_full.groupby("sku_id")["units_sold"]
    pure_seasonal = grp.shift(season_length)
    trailing_avg = grp.transform(
        lambda s: s.shift(1).rolling(4, min_periods=1).mean())
    weekly_full["baseline_forecast"] = pure_seasonal.where(
        pure_seasonal.notna() & (pure_seasonal > 0), trailing_avg)
    return weekly_full
```

15.3 Risk Scoring Logic (src/risk_scoring.py)

```
risk["lead_time_weeks"] = risk["lead_time_days"] / 7
risk["demand_during_leadtime"] = (
    risk["weekly_demand_rate"] * risk["lead_time_weeks"])
risk["projected_position"] = (
    risk["on_hand_units"] + risk["on_order_units"]
    - risk["demand_during_leadtime"])
risk["stockout_risk_score"] = np.clip(
    1 - (risk["projected_position"] /
         risk["reorder_point"].replace(0, np.nan)).fillna(1), 0, 1)

risk["demand_8wk"] = risk["weekly_demand_rate"] * 8
risk["overstock_ratio"] = (
    risk["on_hand_units"] / risk["demand_8wk"].replace(0, np.nan))
risk["overstock_risk_score"] = np.clip(
    (risk["overstock_ratio"] - 1) / 3, 0, 1).fillna(0)
```

16. Appendix

16.1 Glossary

Term	Meaning
SKU	Stock-Keeping Unit -- a single sellable product variant
Stockout	Running out of stock; demand cannot be met, resulting in lost sales
Overstock	Holding far more stock than will sell soon; locks up capital
Lead time	Time between placing a supplier order and it arriving, ready to sell
Reorder point	The stock level at which a replenishment order should be placed
Baseline	A simple reference forecast that a real model must outperform to be worth using
Backtest	Testing a forecast against history it did not train on, to estimate real accuracy
WAPE	Weighted Absolute Percentage Error -- the primary accuracy metric used in this project
Rolling-origin CV	Backtesting method that repeatedly trains on the past and tests on the next period
Star schema	One fact table joined to descriptive dimension tables

16.2 Full Data Dictionary

sales_daily

Column	Description
date	Calendar date of the sales record
sku_id	Product identifier (joins to sku_master)
units_sold	Units sold that week/day
revenue	Revenue for that SKU-period
unit_price	Average selling price that period
promo_flag	1 if inferred to be on promotion, else 0

sku_master

Column	Description
sku_id	Product identifier (primary key)
description	Most frequent product description
category / subcategory	Product classification (keyword-derived)
launch_date	First recorded sale date (proxy for launch)
unit_cost / list_price	Estimated cost and observed list price

calendar

Column	Description
date	Calendar date (primary key)
week / month / season	Time attributes for seasonality
is_holiday	UK holiday indicator
promo_event	Named promotional window, if any

inventory_snapshots

Column	Description
date / sku_id	Snapshot date and product
on_hand_units	Simulated stock physically available
on_order_units	Simulated stock ordered, not yet received
lead_time_days	Simulated supplier replenishment lead time
reorder_point	Simulated stock level that should trigger a reorder

16.3 References

- Online Retail II dataset -- UCI Machine Learning Repository: archive.ics.uci.edu/dataset/502/online+retail+ii
- scikit-learn documentation: scikit-learn.org
- Streamlit documentation: docs.streamlit.io
- FastAPI documentation: fastapi.tiangolo.com

16.4 Repository & Live Links

- GitHub repository: <https://github.com/Sami21234/ZIDIO-PROJECTS>
- Live dashboard: <https://zidio-foresight.streamlit.app/>
- Scoring API: <https://foresight-scoring-api-xz0j.onrender.com>

17. Sample Data Extracts

The following are genuine sample rows drawn directly from the processed output tables, included so the schema's actual shape can be inspected without opening the CSV files.

17.1 sales_daily (fact table)

date	sku_id	units_sold	revenue	unit_price	promo_flag
2009-12-01	10002	12	10.20	0.85	0
2009-12-03	10002	6	5.10	0.85	0
2009-12-04	10002	73	62.05	0.85	0
2009-12-06	10002	49	41.65	0.85	0

17.2 sku_master (dimension)

sku_id	description	category	unit_cost	list_price
10002	INFLATABLE POLITICAL GLOBE	General Merchandise	0.42	0.85
10002R	ROBOT PENCIL SHARPENER	Stationery & Cards	2.84	5.17
10080	GROOVY CACTUS INFLATABLE	General Merchandise	0.20	0.39
10109	BENDY COLOUR PENCILS	Stationery & Cards	0.23	0.42

17.3 calendar (dimension)

date	week	month	season	is_holiday	promo_event
2009-12-01	49	12	Winter	0	Christmas Sale
2009-12-02	49	12	Winter	0	Christmas Sale
2009-12-03	49	12	Winter	0	Christmas Sale

17.4 inventory_snapshots (dimension, simulated)

date	sku_id	on_hand_units	on_order_units	lead_time_days	reorder_point
2009-12-07	10002	1140	0	8	742
2009-12-14	10002	1081	0	8	742
2009-12-21	10002	1069	0	8	742

18. Acceptance Criteria Compliance Matrix

The engagement brief specifies explicit acceptance criteria for each deliverable (D1-D7). This matrix maps each criterion to where it was satisfied in this project, providing a direct audit trail for grading.

18.1 D1 - Data Pipeline

Criterion	Evidence
Ingests all four extracts, produces one analysis-ready dataset	src/pipeline.py, Section 3
Cleaning steps coded, not manual	src/pipeline.py functions, Section 4.2
Re-runs end-to-end from raw files with a single command	python -m src.pipeline (Section 6.4)
Key cleaning decisions documented with rationale	Section 4.2, 4.4

18.2 D2 - Data-Quality & EDA Insight Memo

Criterion	Evidence
Reports data-quality issues and handling	Section 4.1, 4.2, 11.1
Shows demand patterns: seasonality, top movers, dead stock	Section 5.3, 5.4, 5.5
States 3+ business-relevant insights in plain language	Section 5.7
Charts labelled, readable by non-technical reader	Figures 1-8, Section 5

18.3 D3 - Demand Forecast Model

Criterion	Evidence
Weekly SKU-level forecast over defined horizon	Section 7.3, 7.4
Seasonal-naïve baseline included	Section 7.2
Rolling-origin CV, WAPE vs baseline reported	Section 7.5
No data leakage	Section 7.3 (shift-before-rolling), 11.2

18.4 D4 - Risk Scoring

Criterion	Evidence
Scores stockout/overstock risk for every SKU	Section 8.1, 8.2, 8.4
Attaches recommended action and value at stake	Section 8.3, 8.5
Logic transparent, explainable	Section 8.1, 8.2, 15.3
Reconciles with decisioning grid	Section 8.3, 8.4

18.5 D5 - Planning Dashboard

Criterion	Evidence
Filterable by category/SKU, shows forecast vs actual and risk	Section 9.2
Surfaces prioritised reorder/markdown list	Section 9.2 (Risk & Decisioning page)
Usable by a non-technical person	Section 9.3
Live and deployed	Section 9.1

18.6 D6 - Deployed Scoring Service

Criterion	Evidence
Returns forecast + risk for a SKU or batch	Section 10.2, 10.3
Documented inputs and outputs	Section 10.2, 10.3
Handles bad input gracefully	Section 10.4
Hosted and reachable via a public URL	See Section 10.1 for current access method

18.7 D7 - Executive Readout

Criterion	Evidence
Leads with rupee/monetary impact and recommended actions	reports/executive_readout.md, Section 12
Explains accuracy and limitations honestly	Section 7.6, Section 13
No unexplained jargon	reports/executive_readout.md

19. Project Risk Register

Risks identified at the outset of the engagement, and how each was mitigated in practice.

Risk	Mitigation Applied
Sparse history for new/low-volume SKUs	Rolling statistics and category-level features provide a fallback signal; low-volume WAPE reported honestly rather than hidden (Section 7.6)
Promotions distort raw demand signal	promo_flag engineered as an explicit feature and validated via bivariate analysis before being trusted (Section 5.2)
Overfitting / data leakage	Strict rolling-origin backtesting; every lag/rolling feature shifted before windowing (Section 7.3, 7.5)
Falling behind the 4-week/time-constrained schedule	Core modelling and risk scoring (Weeks 3) protected first; polish deferred where necessary
Dependency/version mismatches breaking deployment	Exact library versions pinned in requirements.txt after a real version-mismatch incident (Section 11.4)

20. Additional Code Listings

20.1 Feature Engineering (src/features.py)

```
for lag in [1, 2, 4, 8, 52]:
    df[f"lag_{lag}"] = grp.shift(lag)

# shift(1) BEFORE rolling so the current week never leaks
# into its own rolling statistic
shifted = grp.shift(1)
df["roll_mean_4"] = shifted.groupby(df["sku_id"]).transform(
    lambda s: s.rolling(4, min_periods=1).mean())
df["roll_mean_8"] = shifted.groupby(df["sku_id"]).transform(
    lambda s: s.rolling(8, min_periods=1).mean())
```

20.2 Rolling-Origin Backtest (src/rolling_cv.py)

```
for fold in range(n_folds):
    test_end_idx = last_idx - fold * n_test_weeks
    test_start_idx = test_end_idx - n_test_weeks + 1
    train_end_idx = test_start_idx - 1

    train = model_df[model_df["week"].isin(all_weeks[:train_end_idx+1])]
    test = model_df[model_df["week"].isin(
        all_weeks[test_start_idx:test_end_idx+1])]

    model = train_model(train)
    preds = predict(model, test)
    model_wape = wape(test["target"], preds)
    baseline_wape = wape(test["target"],
        test["lag_52"].fillna(test["roll_mean_4"]))
```

20.3 FastAPI Scoring Endpoint (service/main.py)

```
@app.get("/score/{sku_id}")
def score_sku(sku_id: str):
    row = risk_df[risk_df["sku_id"] == sku_id]
    if row.empty:
        raise HTTPException(
            status_code=404,
            detail=f"SKU '{sku_id}' not found in catalog.")
    r = row.iloc[0]
    return {
        "sku_id": sku_id,
        "description": r["description"],
        "weekly_demand_forecast": round(r["weekly_demand_rate"], 2),
        "on_hand_units": int(r["on_hand_units"]),
        "quadrant": r["quadrant"],
        "value_at_stake_gbp": round(r["value_at_stake"], 2),
    }
```

21. Key Formulas Reference

21.1 WAPE (Weighted Absolute Percentage Error)

```
WAPE = sum(|actual - forecast|) / sum(actual)
```

Used instead of MAPE because it divides by the TOTAL actual demand once, rather than dividing each individual row by its own actual value -- avoiding instability when many rows have near-zero demand (see Section 5.5, Section 7.1).

21.2 Safety Stock / Reorder Point

```
reorder_point = (avg_daily_demand * lead_time_days)
                + (z * std_daily_demand * sqrt(lead_time_days))
```

Where z is the safety factor corresponding to a target service level ($z = 1.65$ for approximately 95% service level, used throughout this project). This is the standard operations-research formula for buffering against demand uncertainty during the replenishment lead time.

21.3 Order-Up-To Level

```
order_up_to = reorder_point + (avg_daily_demand * 14)
```

Adds roughly two additional weeks of average demand coverage on top of the reorder point, used in the inventory simulation described in Section 4.5.

21.4 Overstock Ratio

```
overstock_ratio = on_hand_units / (weekly_demand_rate * 8)
overstock_risk_score = clip((overstock_ratio - 1) / 3, 0, 1)
```

Compares current on-hand stock to an 8-week forward demand estimate; a ratio near or below 1 indicates healthy stock, while a ratio well above 1 indicates overstock (Section 8.2).

22. Testing & Validation Summary

Every component of this project was independently validated against real, computed output rather than assumed correct once code executed without error. This section summarizes the checks performed.

Component	Validation Performed	Result
Raw data ingestion	Row counts, dtypes, null counts, negative-value counts checked before and after cleaning	Confirmed 1,067,371 to 1,003,417 rows; zero anomalies remaining post-clean
Pipeline output	sku_master vs sales_daily SKU counts cross-checked; duplicate (date,sku) pairs checked	0 duplicates; counts reconciled
Weekly aggregation	Total units_sold before/after reindexing to a full SKU x week grid compared	Totals matched exactly (11,188,124 units both before and after)
Baseline WAPE	Investigated an initial NaN result; root-caused to a week-anchor mismatch (Section 11.2)	Fixed; WAPE 1.217 confirmed reproducible
Model backtest	4-fold rolling-origin CV; model vs baseline compared per fold, not only on average	Model beat baseline in every individual fold
Risk scores	Top overstock SKUs cross-checked against independently computed EDA dead-stock list	Same SKUs appeared in both analyses, corroborating the finding
API error handling	Tested with both a valid SKU ID and a deliberately invalid one	Valid SKU returned correct JSON; invalid SKU returned a clean HTTP 404, no crash
Dashboard	Verified each of the 6 pages renders with real data and that filters function as expected	Confirmed via direct inspection of running application

23. Submission Checklist

Per Section 13 of the engagement brief, the following items constitute the complete submission package.

Item	Status
Git repository (pipeline, notebooks, model code)	Complete
Live dashboard URL	Complete -- https://zidio-foresight.streamlit.app/
Live/accessible scoring-service	Complete -- see Section 10.1 for access method
README (problem, data, setup, backtest result, assumptions)	Complete
Executive readout	Complete -- reports/executive_readout.md
Data-quality & EDA memo	Complete -- reports/eda_insight_memo.md, reports/data_cleaning_report.md
This final written report (40+ pages)	Complete
3-5 minute demo video	To be recorded -- see accompanying video script
Completed cohort submission form with all links	To be submitted

24. Team & Individual Contributions

[Edit this section to reflect your actual team structure. A suggested format follows.]

Name	Primary Contribution
Mohd Sami	Data pipeline, cleaning, dashboard, API, deployment, and this report
Kalaiyarasi T	EDA
Adarsh Kumar	Forecasting model
Priyanka Kadwa	Risk scoring

This engagement was approached as a team effort covering the full analytics lifecycle described throughout this report: from raw data inspection through to a deployed, stakeholder-facing product, consistent with the brief's framing of the intern as "the data scientist Zidio has placed on a client account."

25. Detailed Work Log

A day-by-day account of work performed across the engagement, mirroring the 20-working-day structure specified in the brief's Section 10.

Week 1 -- Data Foundation

Day	Activity
1	Read the engagement brief; restated the business problem and success metrics; set up the working environment
2-3	Located and downloaded the Online Retail II source data; profiled raw columns, dtypes, and obvious data-quality issues
4-5	Wrote the cleaning pipeline (src/pipeline.py); documented each cleaning decision with before/after row counts

Week 2 -- EDA & Baseline

Day	Activity
6-7	Performed univariate and bivariate analysis: demand distribution, category counts, price-vs-demand, promo effect
8	Performed multivariate analysis: category-by-month heatmap, full correlation matrix; investigated the is_holiday NaN finding
9	Engineered lag, rolling, calendar, and promotion features (src/features.py), enforcing no data leakage
10	Built the seasonal-naive baseline (src/baseline.py); diagnosed and fixed a week-anchor alignment bug that had produced a NaN WAPE; investigated and explained the resulting high WAPE (1.271, later 1.217 with a fallback)

Week 3 -- Modelling & Risk

Day	Activity
11-12	Trained the demand forecast model (src/model.py) on engineered features
13	Implemented rolling-origin backtesting (src/rolling_cv.py); confirmed no leakage in the feature pipeline
14	Evaluated the model against the baseline across 4 folds; performed error analysis by category and volume tier (src/error_analysis.py)

15	Built the stockout/overstock risk-scoring logic and decisioning grid (src/risk_scoring.py); cross-validated top findings against the EDA dead-stock list
----	--

Week 4 -- Productize & Present

Day	Activity
16-17	Built the six-page Streamlit planning dashboard (app/dashboard.py)
18	Built and tested the FastAPI scoring service (service/main.py); diagnosed and resolved a scikit-learn version mismatch during deployment testing
19	Quantified business impact in monetary terms; drafted the executive readout for non-technical stakeholders
20	Final QA against every deliverable's acceptance criteria; assembled this final written report; prepared the demo video

26. Environment & Reproducibility

Per the brief's reproducibility requirement, a grader should be able to clone the repository, follow the README, and re-run the pipeline to the same headline numbers. The following details support that.

Item	Detail
Python version	3.12
Key library versions	Pinned exactly in requirements.txt (see Section 11.4 for why this matters)
Random seed	Fixed (seed = 42) across data simulation and model training for reproducible results
Data source snapshot	Online Retail II, as published on the UCI Machine Learning Repository
Full re-run command sequence	See README.md "Setup & Run" section, reproduced in Section 6.4 of this report

This report, and the analysis it documents, was produced as part of the Zidio Development Data Science & Analytics internship program, engagement: Project FORESIGHT for NorthBay Living.

27. Document Revision History

Version	Date	Changes
1.0	[Date]	Initial complete report covering all engagement deliverables D1-D7

End of report.